

Verifier-Assisted versus One-Shot LLM Intent→Configuration: A Paired NetOpsTraceBench Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed NetOpsTraceBench, a paired comparison of a one-shot LLM intent→configuration pipeline and a verifier-assisted generate-validate-repair pipeline on 1,200 intent→config episodes. Using gpt-5-mini, a deterministic NetOps validator, and Mininet-style emulation, we applied a preregistered binary episode success rule (validator accepts final config AND emulator shows no availability degradation above tolerance AND no automatic rollback). Of 1,200 planned paired tasks we completed 1,199 pairs. Baseline success = 1,196/1,199 (0.9974979149); guarded success = 1,199/1,199 (1.0000000000). The paired-success-rate difference (guarded - baseline) = 0.0025020851 (95% bootstrap percentile CI [0.0000, 0.0058381985], bootstrap_seed=1729, resamples=10,000); exact McNemar two-sided $p = 0.25$. The preregistered power target sought a 5 percentage-point minimum detectable effect; given the observed near-ceiling baseline, the study lacked power to detect much smaller absolute gains. Representative validator traces and provider audit logs show repair was invoked only three times. Under these controlled emulation and task-corpus conditions we find no statistically significant advantage for the verifier-assisted pipeline; we report artifacts, analytic deviation (Wilcoxon → exact McNemar for binary outcomes), and detailed execution accounting to support reproducible interpretation.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent expressed in natural language into executable device configurations for network operations (NetOps). Systems and survey literature emphasize that operational reliability depends critically on orchestration, deterministic validators, and assurance infrastructure rather than on an LLM alone [1,2]. Generate-validate-repair (verifier-assisted) architectures are a canonical mitigation strategy: an LLM proposes candidate outputs, an automated verifier checks constraints and safety, and an automated repair loop iterates until the candidate satisfies verifier rules [3–5]. Despite conceptual support, publicly archived, preregistered, paired evidence measuring whether verifier assistance yields an operationally meaningful improvement (rollback-aware binary success) relative to one-shot generation is sparse.

We address this gap with NetOpsTraceBench: a preregistered, paired, emulator-grounded study comparing (A) baseline one-shot NL→config generation and (B) a guarded generate-validate-repair pipeline using the same shared initial candidate. The study asked: Can a reproducible, workflow-centered benchmark of paired intent→config episodes produce an objective paired binary success metric that reliably distinguishes verifier-assisted pipelines from one-shot generation across two

simulated topologies? Our contributions are (1) a preregistered experimental design and full execution artifacts (study ID rX4f7-1); (2) a paired analysis on 1,199 completed pairs using gpt-5-mini and a deterministic validator; (3) transparent reporting of an analytic deviation (final confirmatory test used an exact McNemar test appropriate for paired binary outcomes) and execution accounting that explains why the guarded pipeline rarely required repair.

II. RELATED WORK

Recent surveys and system papers position LLMs within AIOps/NetOps pipelines and emphasize the importance of verifiers, provenance, and end-to-end workflow evaluation [1,6]. Work on verifier-guided NL→formal translations and intent→topology compilation has shown the feasibility of generate-validate-repair approaches in domains such as security topology compilation and CTL/specification generation [3,7]. Empirical and conceptual studies document LLM failure modes relevant to NetOps—hallucination, incorrect semantics/units, overconfidence, provenance loss, and prompt injection—which motivate guarded architectures and runtime auditing [8–10]. Several recent prototypes demonstrate LLM-assisted configuration generation and misconfiguration detection but typically evaluate on constructed or small held-out sets [4,5,11]. That literature motivates workflow-centered evaluation metrics (trace quality, rollback measurement, canaries) rather than one-shot QA metrics; NetOpsTraceBench operationalizes a paired, rollback-aware binary success metric in a preregistered framework and archives the run artifacts for reproducibility.

III. METHODOLOGY

Preregistration and experimental scope: The study design was preregistered under ID rX4f7-1 and frozen prior to execution (preregistration manifest SHA-256 = 77abe8b4...). The preregistration specified confirmatory hypotheses, a single primary estimand (paired_success_rate_difference_guarded_minus_baseline), deterministic inclusion/exclusion rules, contamination thresholds, stopping rules, random seeds, maximum model-call budget, and a complete analysis plan. NetOpsTraceBench_v1 targeted 1,200 paired intent→config tasks partitioned into two simulated topology clusters (edge = 600, core = 600). Task examples were public synthetic or consented; every task

carried deterministic provenance metadata. The preregistered power plan assumed a baseline success near 0.70 and set a minimum detectable effect (MDE) of 5 percentage points (absolute) at two-sided $\alpha=0.05$ and power 0.8; sensitivity scenarios over a range of baseline and discordant-pair assumptions were specified in the preregistration.

Pipelines, transformations, and instrumented components: We compared two paired conditions executed from a single shared initial candidate per pair. Baseline (one-shot) invoked a single LLM generation call to produce an initial candidate configuration. Guarded (verifier-assisted) used the same shared initial candidate and permitted at most one automated repair iteration when the deterministic validator rejected the candidate; preregistration limited `maximum_repair_calls_per_task = 1` and `initial_generation_calls_per_task = 1` to bound model-call budget. The requested model was `gpt-5-mini` (provider recorded as `openai_responses`) invoked through orchestration adapter `hosted_netops_gvr_v1`. `Deterministic_netops_task_generator_v5` produced canonical, vendor-agnostic intent→configuration tasks; `deterministic_netops_validator_v5` performed parsing, intent-constraint validation, dependency checks, and emulator preflight evaluation. The execution contract capped `maximum_model_calls = 2,400` and defined the `scientific_confirmatory` execution mode.

Inclusion, contamination, and stopping rules: Inclusion required that neither episode in a pair be deterministically excluded. The preregistration implemented contamination detection by (a) a normalized substring/token-overlap test with threshold `T_contam = 0.85` and (b) an optional embedding-similarity secondary check (cosine threshold = 0.92) for flagged items. If either test exceeded thresholds the pair was flagged and excluded from the primary confirmatory set (but retained for exploratory reporting). Pairs were also excluded if both episodes failed to produce parsable vendor renderings after one automated re-execution attempt. The stopping rule required running the full planned workload unless (i) cumulative model calls reached the `maximum_model_calls` cap, or (ii) automation halted due to deterministic component failure after the one allowed automated recovery attempt; all such events were logged.

Determinism, seeds, and failure handling: The preregistration froze generation seeds and analysis seeds. Deterministic task generation and validator behavior reduce sampling variance from non-deterministic test artifacts; model nondeterminism remained possible at the provider side but was bounded by the single-call-per-stage policy. Missingness handling followed the preregistered plan: the primary analysis used `complete_pair_primary` (exclude flagged/excluded pairs); a sensitivity analysis treated excluded or failed episodes as failures (`complete_pair_primary_with_failure_as_zero_sensitivity`). All exclusion reasons, failed episode counts, and provider audit traces were archived to preserve transparent accounting.

Success metric, confirmatory test, and bootstrap CI: The

preregistered binary episode success rule required three conjunctive outcomes: (1) final configuration accepted by the deterministic validator (parsing and intent constraints satisfied), (2) emulator execution shows availability degradation no greater than the preregistered tolerance, and (3) no automatic rollback event occurred during simulated execution. Although the preregistration specified a paired nonparametric Wilcoxon signed-rank test, the archived outcomes were binary for each episode. Following the preregistered emphasis on correct paired-outcome inference, the final archived confirmatory analysis applied an exact McNemar two-sided test to the 2×2 paired contingency table and computed a paired bootstrap percentile 95% confidence interval for the paired-success-rate difference using frozen `bootstrap_seed = 1729` and 10,000 resamples. This analytic deviation (Wilcoxon → exact McNemar) is recorded in the analysis artifacts and in the Disclosure Statement.

Instrumentation, auditing, and artifact recording: The run captured provider call audit metadata (`hosted_model_call_event_count` and `stage_counts`), all raw model outputs, shared initial candidate traces, repair provider traces when present, validator traces (validation_before/validation_after), emulator execution logs, `task_manifest` entries, `transformation_manifest`, and a full hash manifest. Key configured limits recorded in `model_configuration.json` included `max_output_tokens = 1500` and `requested_model = gpt-5-mini`. The execution manifest records `planned_episode_count = 2,400`, `completed_episode_count = 2,398`, `failed_episode_count = 2`, and `model_calls_used = 1,203`. All seeds, analysis code (`paired_binary_analysis_v1`), and final analysis artifacts (`analysis/paired_contingency_table.csv`, `analysis/results.json`, `analysis/paired_difference_figure.svg`) are archived to enable exact reproduction.

Predefined sensitivity and exploratory plans: The preregistration enumerated exploratory analyses that the archived run executed or preserved for downstream work: (a) distribution of repair iterations and their correlation with final success and time-to-repair; (b) automated failure-mode categorization (semantic unit errors, missing dependencies) derived from deterministic parsers and emulator checks; (c) sensitivity of the primary binary success to alternative availability-degradation tolerances and to treating flagged contamination pairs as failures. All exploratory outputs were labeled as such and are reported separately from the confirmatory inference to control Type I error per the preregistered multiplicity plan.

IV. RESULTS

Execution and primary accounting (archived artifacts). The run attempted the preregistered 2,400 episodes (1,200 paired tasks) and completed 2,398 episodes (`completed_pairs = 1,199`; `failed_episode_count = 2`). Provider audit fields recorded `hosted_model_call_event_count = 1,203`, `completed_hosted_model_call_event_count = 1,202`, and `failed_hosted_model_call_event_count = 1`. Stage-level accounting shows `shared_initial_generation`

= 1,200 and repair = 3 (execution/provider audit and deterministic_reconciliation.json). Artifact_count for the full run = 8,408 and the run completed at 2026-08-30T03:15:12.141890+00:00 (execution manifest).

Primary confirmatory outcomes (archived analysis outputs). Using the preregistered binary success rule, the archived condition summary (analysis/condition_summary.csv) reports baseline_success_count = 1,196 of 1,199 complete pairs (baseline_success_rate = 0.9974979149291076) and guarded_success_count = 1,199 of 1,199 (guarded_success_rate = 1.0). The paired contingency table (analysis/paired_contingency_table.csv) is $n_{11} = 1,196$ (both succeeded), $n_{10} = 3$ (guarded succeeded, baseline failed), $n_{01} = 0$ (baseline succeeded, guarded failed), $n_{00} = 0$ (both failed). The point estimate for the paired success-rate difference (guarded - baseline) = 0.002502085070892446. The archived confirmatory exact McNemar two-sided p-value = 0.25. The paired bootstrap percentile 95% confidence interval computed with bootstrap_seed = 1729 and 10,000 resamples is [0.0000, 0.005838198498748957] (analysis/results.json and analysis/analysis_log.jsonl).

Missingness and failed-call accounting. The preregistered missingness summary and deterministic reconciliation artifacts report complete_pair_count = 1,199, failed_baseline_episodes = 1, failed_guarded_episodes = 1, and both_missing_pairs = 1 (analysis/missingness_summary.csv; deterministic_reconciliation.json). No pairs were excluded for contamination under the preregistered thresholds (analysis/contamination_summary.csv empty). The run therefore uses the complete_pair_primary treatment as planned.

Repair invocation and provider audit diagnostics. Repair-stage calls were invoked only three times (stage_counts.repair = 3 in provider_call_audit). Provider audit also shows no cross-condition cache reuse (cross_condition_cache_reuse_observed = false) and confirms the intended shared_initial_generation semantics (shared initial candidate used for paired episodes). The low repair count is corroborated by the scoring artifacts: representative scoring JSONs (e.g., execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json) show validator_trace.validation_after.valid = true and validator_trace.repair_applied = false for typical episodes. In other words, the deterministic validator accepted the initial shared candidate in the vast majority of tasks, leaving only three guarded episodes that required a repair call.

Representative per-task diagnostics (artifact-grounded). Representative task_manifest entries and scoring traces illustrate typical episode structure and validator acceptance criteria. For example, pair-000001 (execution/scoring/task-000001-baseline.json) had parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, and validation_after.normalized_configuration matching the reference; validation_after.valid = true and repair_applied = false. A harder task example (pair-000002) shows dif-

ficulty.level = "hard" with changed_interface_count = 2 and required_command_count = 2, yet its initial candidate was still validated as valid by the deterministic validator. These per-episode fields (parsed_command_count, required_command_count, difficulty pattern) are recorded in the representative scoring artifacts and demonstrate that many tasks contained explicit required sequences which the generator reproduced correctly.

Interpretation of discordant pairs and CI. The observed discordant pattern ($n_{10} = 3$, $n_{01} = 0$) implies that all discordant evidence favored the guarded pipeline; however, the absolute number of discordant pairs is tiny relative to the total sample. The bootstrap CI lower bound is exactly 0.0000 (percentile method) and the upper bound ≈ 0.00584 , indicating that the data are compatible with a small positive absolute gain but not with the preregistered target MDE (5 percentage points). The exact McNemar $p = 0.25$ formalizes that, under these discordant counts, the test does not reject the null of no paired difference at $\alpha = 0.05$.

Why repair was rarely exercised. Two archived facts explain the infrequent repair calls: (1) the synthetic task generator produced tasks with explicit required sequences and constrained allowed field changes (deterministic_netops_task_generator_v5 task_payloads record required_sequence entries), increasing the chance that a straightforward canonical candidate satisfies intent constraints; (2) the deterministic validator configuration (deterministic_netops_validator_v5) accepted initial candidates when all intent constraints and preflight checks matched the rendered candidate. Provider audit stage_counts (shared_initial_generation = 1,200, repair = 3) and scoring artifacts jointly document these operational mechanisms and support the conclusion that the guarded pipeline had little opportunity to improve over an already-high baseline under the chosen corpus and validator strictness.

Ancillary archived metrics. The analysis artifact set includes analysis/paired_difference_figure.svg (visual CI and point estimate), analysis/paired_contingency_table.csv (full contingency counts), and analysis/analysis_log.jsonl (bootstrap parameters: resamples = 10,000; seed = 1729). The execution manifest archives timing, provider call counts, and the preregistration SHA-256 reference; these artifacts are sufficient to reproduce the confirmatory estimates and the diagnostic counts reported above.

V. DISCUSSION

Statistical and operational interpretation: The exact McNemar test ($p = 0.25$) and the paired bootstrap CI [0.0000, 0.0058381985] both indicate the observed guarded advantage is not distinguishable from zero under our confirmatory criterion. The empirical discordant counts ($n_{10} = 3$, $n_{01} = 0$) directly determine the paired estimate: paired_difference = $3/1,199 = 0.0025020851$. Because the preregistered MDE was 0.05 (5 percentage points), the study—while sized for that target—was not powered to detect the much smaller absolute difference we observed. Concretely, observing a 5 pp absolute

paired difference on the realized `complete_pair_count` (1,199) would require approximately 60 discordant pairs ($0.05 \times 1,199 \approx 59.95$), but we observed only three discordants; this arithmetic shows why the planned MDE and realized discordant count are inconsistent with detecting the tiny observed effect.

Artifact-grounded diagnostics explain why repairs were rare. The provider audit and stage counts record only three repair invocations (`repair = 3`) and `model_calls_used = 1,203`, corroborated by the validator traces where `validation_after.valid = true` and `repair_applied = false` for representative tasks (see `execution/scoring/task-000001-baseline.json`). Two interacting design choices reduced headroom for verifier benefit: (1) the synthetic task generator produced well-specified intents with canonical `required_sequences` (`deterministic_netops_task_generator_v5`), and (2) the deterministic validator (`deterministic_netops_validator_v5`) used parsing and constraint checks that accepted many reasonable initial candidates. These two factors are visible in representative `task_manifest` entries (`required_sequence` and `difficulty` fields) and in the scoring artifacts where `parsed_command_count == required_command_count` and `violation_count == 0` for typical episodes. Together they explain the low frequency of repair-triggered iterations and the near-ceiling baseline success ($\approx 99.75\%$).

Reproducibility and forensic trace evidence: All numerical claims above are reproducible from archived artifacts. The confirmatory analysis used `bootstrap_seed = 1729` with 10,000 resamples to compute the percentile CI (`analysis/analysis_log.jsonl`), and the paired contingency table is published (`analysis/paired_contingency_table.csv`). Execution accounting (`execution/execution_manifest.json`) reports `started_at_utc` and `completed_at_utc`, `model_calls_used`, `completed_pairs`, and `failed_episodes`. `Artifact_count = 8,408` and representative scoring JSON entries include `validator_trace` objects showing `validation_before/after`, `observed_touched_field_count`, and `repair_applied` flags. Those deterministic traces permit exact reproduction of the numeric summaries without human adjudication.

Threats to validity made concrete by artifacts: Internal validity is high because task generation, seeds, and deterministic validators were frozen in the preregistration (preregistration SHA-256 = 77abe8b4...5712) and automation enforced exclusion rules; the `missingness_summary.csv` documents the two failed episodes and the single `both_missing_pair`. Construct validity depends on how well the binary episode success rule (`validator acceptance + emulator availability tolerance + no rollback`) maps to operator-valued correctness; the validator’s acceptance thresholds and the emulator’s availability tolerance are parameters embodied in the validator traces and `transformation_manifest` and therefore materially affect measured outcomes. External validity is limited: only one model family (`requested_model = gpt-5-mini`) was invoked (`execution/model_configuration.json`), tasks were synthetic/consented and vendor-agnostic, and emulation is Mininet-style rather than vendor-specific hardware; these constraints appear in `task_manifest` entries and in the emulator preflight logs. Con-

clusion validity is constrained by the small number of discordant pairs: standard paired tests rely on sufficient discordance to support inference, and here the discordant mass (3/1,199) is too small to yield narrow hypothesis tests for small absolute differences.

Implications for benchmark design and operational practice: The artifacts show that when initial generation already satisfies deterministic verifier rules for well-specified synthetic tasks, guarded architectures will seldom be exercised, and measured binary success gains will be tiny. To reveal verifier value in future studies, designers should (a) increase nominal task ambiguity or adversarial content so that initial candidates more frequently violate intent constraints, (b) vary validator strictness to control repair invocation rates, and (c) test multiple model families or fine-tuned variants to measure model-validator interactions. The archived run provides concrete knobs: change `task_manifest` difficulty fields, adjust validator violation thresholds, or expand `maximum_repair_calls_per_task` to study repair dynamics—each change is reproducible because seeds, manifests, and validator traces are preserved.

Operational diagnostics and sensitivity: The low repair count (3) correlated with specific task patterns (deterministic task difficulty levels labelled "medium" or "hard" in representative manifests) but was insufficient to permit reliable subgroup inference. Exploratory artifacts (`analysis/condition_summary.csv` and `analysis/missingness_summary.csv`) can be used to stratify by difficulty and re-estimate discordant rates; however, such exploratory subgroup tests remain underpowered without increasing task diversity or sample size. The run’s provider audit also shows a single failed `hosted_model_call_event_count = 1`; recovery and retry rules are logged and the `stopping_rule` was enforced without manual intervention.

Concluding operational guidance: The null confirmatory result—guarded advantage not statistically significant—should be interpreted as conditional on the task corpus, validator configuration, and requested model. The run artifacts consistently point to limited headroom (near-ceiling baseline) and low repair invocation as the proximate causes. For practitioners, the key takeaway is not that verifiers are universally unnecessary, but rather that their measured value depends strongly on task ambiguity and validator policy. We provide the full artifact set (task manifests, validator traces, provider audits, analysis logs, and seeds) to enable sensitivity experiments that vary these parameters and thereby expose conditions under which verifier scaffolding yields operationally meaningful improvements.

VI. LIMITATIONS

- Single model family tested (`gpt-5-mini`); results do not imply generality across other LLMs or fine-tuned variants.
- Task corpus skew: synthetic/consented tasks yielded near-ceiling baseline success ($\approx 99.75\%$), limiting head-

room and statistical power to detect practical improvements by guarded pipelines.

- Emulator fidelity: Mininet-style emulation and deterministic validators approximate production behavior but may not capture all deployment failure modes (routing convergence, vendor idiosyncrasies, transient telemetry).
- Verifier configuration dependence: deterministic_netops_validator_v5 acceptance thresholds and parsing rules materially affect outcomes; different verifier strictness could change repair invocation rates and measured gains.
- Analytic deviation documented: the preregistration specified a Wilcoxon signed-rank paired procedure; the archived confirmatory analysis used an exact McNemar test because outcomes were binary. This deviation is recorded in the archived analysis artifacts and in the Disclosure Statement above.
- Operational external validity: absence of heterogeneous vendor renderers, real production templates, and live rollouts constrain direct production generalization.

VII. CONCLUSION

We preregistered and executed NetOpsTraceBench, a paired, emulator-grounded study comparing one-shot and verifier-assisted NL→configuration pipelines on 1,199 completed pairs. Under our synthetic task corpus, deterministic validator settings, and gpt-5-mini as requested, baseline one-shot generation already achieved near-ceiling success ($\approx 99.75\%$). The guarded pipeline increased absolute success by ≈ 0.25 percentage points (paired difference = 0.0025020851) with exact McNemar two-sided $p = 0.25$ and a 95% bootstrap CI that includes zero. Archived execution manifests, provider audits (model_calls_used = 1,203; repairs invoked = 3), representative validator traces, and analysis artifacts support the interpretation that the observed null result reflects limited headroom given task and verifier choices rather than definitive evidence that verifiers are never useful. Future work should intentionally design more challenging and adversarial task sets, vary verifier strictness, and test multiple LLM families to determine conditions under which verifier scaffolding yields operationally meaningful improvements. All run artifacts, analysis code, seeds, and logs are archived to enable exact reproduction and follow-up sensitivity studies (see Disclosure Statement).

DISCLOSURE STATEMENT

Mandatory disclosure (preregistered run rX4f7-1). LLM(s) and provider: requested model = gpt-5-mini via provider recorded as openai_responses; model configuration archived at execution/model_configuration.json (requested_model=gpt-5-mini, max_output_tokens=1500). Orchestration/adaptor: hosted_netops_gvr_v1 executed the paired benchmark. Deterministic tooling: deterministic_netops_validator_v5 performed canonical parsing, intent-constraint validation, and emulator preflight checks; deterministic_netops_task_generator_v5 produced

synthetic tasks. Exact initial master prompt: archived at provenance/master_prompt.txt (immutable reference SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5a cdf1582bb39b15ba). Topic constraint and autonomy boundary: the human Research Director selected the topic family and approved the master prompt pre-lock; after the autonomy lock the AI autonomously formulated the research question, conducted literature review, generated the preregistration, executed the experiment, performed analysis, and wrote and revised the manuscript. Preregistration: NetOpsTraceBench (ID rX4f7-1) preregistration manifest SHA-256 = 77abe8b4 93bde44484e0b3d02cfd7f9ec1621ff7e7847889082ae177 a7ed5712; preregistered design (confirmatory hypotheses, inclusion/exclusion, seeds, stopping rules, analysis plan) was frozen before execution. Execution summary: started 2026-08-30T01:21:55.415955+00:00, completed 2026-08-30T03:15:12.141890+00:00; planned episodes 2,400; completed episodes 2,398; completed pairs 1,199; model_calls_used = 1,203; repair-stage calls observed = 3. Analysis artifacts and deviation record: the preregistration specified a Wilcoxon signed-rank paired procedure; the archived executed confirmatory analysis used an exact McNemar two-sided test on paired binary outcomes plus a paired bootstrap percentile CI. This post-preregistration analytic choice is documented in the archived analysis artifacts (analysis/paired_contingency_table.csv, analysis/results.json, analysis/analysis_log.jsonl, analysis/paired_difference_figure.svg) produced as part of the run that completed at 2026-08-30T03:15:12.141890+00:00. Artifacts and reproducibility: raw model outputs, provider traces, validator traces, task_manifest, transformation_manifest, execution logs, analysis code, and all seeds are archived; artifact_count = 8,408 and full hash manifest path = execution/execution_manifest.json. Human involvement: pre-lock collaboration and infrastructure development occurred; no human manual editing, selective revision, or ad hoc text insertion occurred on the manuscript content after the autonomy lock—the AI performed internal autonomous revision cycles only. Technical restarts and deviations: execution manifest records two failed episodes and the analytic deviation described above; all deviations are logged in execution/execution_log.jsonl. The disclosure above is complete relative to the archived artifacts supplied with this run.

REFERENCES

- [1] <http://hdl.handle.net/20.500.12708/229494>
- [2] <https://arxiv.org/abs/2608.13389>
- [3] <http://arxiv.org/abs/2403.09369>
- [4] <https://doi.org/10.2139/ssrn.6947162>
- [5] <https://doi.org/10.2139/ssrn.7222278>
- [6] <https://doi.org/10.20935/acadai8260>
- [7] <https://doi.org/10.3390/info17030297>
- [8] <https://doi.org/10.3390/app16010085>